

TradeUp: Pakistani Stock Exchange Trading Simulator

SPROJ Report

Deployment: <https://p04-trade-up.vercel.app/>



Backend: <http://136.119.205.38/api/docs>

M. Umar Zubair (26100355)

M. Rayyan Khan (26100216)

M. Raiyaan Junaid Hamid (25100137)

M. Ahmad (26100200)

M. Shahmir Sher Qazi (26100204)

Advisor:

Waqar Ahmed

**School of Science and Engineering
Lahore University of Management Sciences**

Submission Date: 31 April 2026

Acknowledgement and Dedication

Certificate

I certify that the senior project titled “**TradeUp**” was completed under my supervision by the following students:

M. Umar Zubair (26100355)

M. Rayyan Khan (26100216)

M. Raiyaan Junaid Hamid (25100137)

M. Ahmad (26100200)

M. Shahmir Sher Qazi (26100204)

and the project deliverables meet the requirements of the program.

Date:

Advisor (Signature)

Date:

Co-advisor (if any)

Table of Contents

- 1. Introduction
 - a. Introduction
 - b. Objective and Scope
 - a. Development Methodology
 - b. Contributions
- 2. System Requirements
 - a. System Actors
 - Functional Requirements
 - 2c. Non-Functional Requirements
- 3. System Architecture
 - a. Architecture Diagram
 - b. Architecture Description
 - d. Justification of the Architecture
 - g. Tools and Technologies
- 4. Requirements Specifications
 - a. Use Cases
 - b. Class Diagram

- c. Sequence Diagrams
 - User Registration
 - User login
 - Place Market Order
 - Place Limit Order
 - View Portfolio
 - AI Assistant
 - Game/Tournament
 - Achievement
 - Educational content
 - Leaderboard Update
 - News Analysis
 - Stock Price Alert
 - Chat
 - Tournament Results
 - Real time stock data sync
- 5. Software Development Methodology and Plan
 - a. Software Process Selection

- 6. Database Design and Web Services
 - a. Database Design
 - b. API Specification
- 7. System User Interface
- 8. Project Security
 - 8a. STRIDE Threat Modeling Summary
 - 8b. Key Security Controls Implemented
 - 8c. OWASP Top 10 Compliance
 - 8d. SonarQube / Static Analysis
- 9. Risk Management
 - Potential Risks and Mitigation Strategies
- 10. Testing and Evaluation
- 11. Deployment Guidelines
- 12. Conclusion
 - a. Summary
 - b. Challenges
 - c. Future
- 13. Review checklist

- 14. References

List of Figures

- Figure 1: System Architecture Diagram
- Figure 2: System Use Case Diagram
- Figure 3: Class Diagram
- Figure 4: Sequence Diagram - User Registration
- Figure 5: Sequence Diagram - User Login
- Figure 6: Sequence Diagram - Place Market Order
- Figure 7: Sequence Diagram - Place Limit Order
- Figure 8: Sequence Diagram - View Portfolio
- Figure 9: Sequence Diagram - AI Assistant
- Figure 10: Sequence Diagram - Game/Tournament Entry
- Figure 11: Sequence Diagram - Achievement
- Figure 12: Sequence Diagram - Educational Content Completion
- Figure 13: Sequence Diagram - Leaderboard Update
- Figure 14: Sequence Diagram - News Analysis
- Figure 15: Sequence Diagram - Stock Price Alert

- Figure 16: Sequence Diagram - Chat
- Figure 17: Sequence Diagram - Tournament Results
- Figure 18: Sequence Diagram - Real Time Stock Data Sync
- Figure 19: Database Entity-Relationship Diagram

1. Introduction

a. Introduction

Describe the application that you are developing. Also discuss:

TradeUp is a full-stack web application that simulates stock trading on the Pakistani Stock Exchange (PSX). It allows users to buy and sell real PSX-listed stocks using virtual currency, with prices sourced from live market data. The platform also integrates AI coaching, community features, news sentiment analysis, candlestick chart views, and a gamified AI-driven tournament system.

1. Domain of your application

Financial Technology (FinTech) — specifically stock market education and trading simulation.

2. Target users of this application

Pakistani investors and finance students who want to learn stock trading without risking real money. Secondary users include finance enthusiasts, university students studying economics or finance, and casual learners interested in the PSX.

3. What have you produced: a mobile app, a web app etc.?

A responsive, browser-accessible web application built with Next.js on the frontend and NestJS-Fastify on the backend, deployable on Vercel (frontend) and GCP Kubernetes and production-ready cloud infrastructure (backend).

b. Objective and Scope

2. The primary objective of TradeUp is to democratize financial literacy in Pakistan by providing a risk-free environment where users can practice real-time stock trading. Pakistan's retail investor base is growing but largely lacks structured learning resources. TradeUp addresses this gap by simulating the PSX environment accurately — using live KSE-100 index data and PSX-listed stocks — while removing the financial risk associated with real trading.

The platform enhances the learning domain through several mechanisms: real-time market data feeds enable users to experience authentic price movements; an AI chatbot coach (powered by Google Gemini) provides personalized portfolio analysis and trade recommendations; a tournament mode allows competitive simulated trading against other users with AI-generated 30-day market trajectories; and a community forum enables peer learning. The scope includes user account management, portfolio management, trade journaling, watchlists, social features (friends, community posts, reactions), and AI-driven analytics.

a. Development Methodology

The project followed an Agile Scrum methodology with four sprints, each delivering a shippable increment of the product. Each sprint included planning, development, security review (STRIDE threat modeling), and retrospective sessions. The team used GitHub for version control, Windsurf (VS Code fork) as the primary IDE, and AWS for cloud infrastructure setup. Generative AI tools (Claude, Gemini) were used to assist in code generation and code review, always under team supervision. Task tracking was managed via GitHub Projects, and sprint deliverables were reviewed against the project rubric before each submission.

b. Contributions

TradeUp differentiates itself from existing platforms like Investopedia Simulator or Moneybhai in the following key ways:

- **PSX-specific:** Unlike US-market simulators, TradeUp exclusively uses live Pakistani Stock Exchange data (KSE-100 index, PSX-listed stocks), making it uniquely relevant to the Pakistani market.

- **AI Tournament Engine:** The Oracle system uses Google Gemini to generate fully synthetic 30-day market trajectories with AI-written news headlines tied to price movements. No comparable Pakistani trading simulator offers AI-generated market scenarios.
- **Integrated AI Coach:** A multi-turn conversational AI coach is context-aware of the user's actual portfolio, watchlist, and live prices — giving truly personalized advice rather than generic tips.
- **Trade Journaling with Sentiment Categories:** Users tag every sell with a reason (Target Hit, Panic/Emotion, Needed Cash), building behavioral awareness of their trading psychology.
- **Community + Social Layer:** Unlike most simulators that are purely transactional, TradeUp includes community posts, reactions, @mention support, friend networks, and block/save features — enabling a social learning ecosystem.
- **News Sentiment Analysis:** Every news headline can be analyzed by AI for sentiment polarity, confidence score, and keywords, helping users practice news-driven trading decisions.

2. System Requirements

This chapter outlines the actors, functional requirements, and non-functional requirements of the TradeUp system. It defines who interacts with the system, what the system must do (functional requirements), and the quality attributes the system must satisfy (non-functional requirements).

a. System Actors

List down the actor names and give a 2-3 lines description of the role of each actor

Actor Name	Description
Guest (Unauthenticated User)	Any visitor to the platform who has not logged in. Can view featured stocks, access public news, and browse the signup/login pages. Cannot trade, post in community, or use AI features.
Registered User	A logged-in user with a virtual trading account. Has full access to portfolio management, trading, watchlists, community features, AI chatbot, news sentiment analysis, and tournaments. The primary actor of the system.
Tournament Participant	A registered user who has joined an active tournament. Can execute tournament buy/sell orders, view the tournament leaderboard and portfolio, and receive AI performance analysis at tournament end.
Admin	A privileged user with elevated access. Can end tournaments, manage users, and has access to admin-

	specific endpoints. Responsible for platform oversight and moderation.
AI System (Gemini)	An external automated actor (Google Gemini API). Provides AI chatbot coaching responses, generates tournament market trajectories and news headlines, performs sentiment analysis on news, and delivers post-tournament performance reviews. Interacts with the backend via API calls.

Functional Requirements

Authentication Module

1. The system shall allow users to register with a unique email, username, password, and gender.
2. The system shall allow registered users to log in and receive a JWT token.
3. The system shall allow authenticated users to update their email, password, and display name.
4. The system shall allow users to upload and retrieve a profile picture.

Trading Module

5. The system shall allow users to buy PSX stocks using their virtual balance.
6. The system shall allow users to sell stocks from their portfolio, recording a sell reason and optional sell note.

7. The system shall compute and display real-time Profit and Loss (PnL) per holding and portfolio-wide.
8. The system shall maintain a paginated transaction history per user.
9. The system shall allow users to add virtual funds to their wallet.

Market Data Module

10. The system shall fetch and display live PSX stock prices and KSE-100 index data.
11. The system shall provide OHLCV candlestick data in multiple timeframes (1m, 5m, 15m, 30m, 1h, 1d).
12. The system shall push real-time stock price ticks to subscribed frontend clients via WebSocket.

Watchlist Module

13. The system shall allow users to add and remove stocks from a personal watchlist.
14. The system shall display a user's watchlist with live prices.

News and Sentiment Module

15. The system shall fetch the latest market news from an external news API.
16. The system shall fetch stock-specific news based on a given ticker symbol.
17. The system shall allow users to request AI-powered sentiment analysis (positive/negative/neutral, confidence, keywords) for any news headline.

Community Module

- 18. The system shall allow authenticated users to create, view, and delete community posts with tags.
- 19. The system shall support nested comment threads on posts.
- 20. The system shall allow users to react to posts with five reaction types: LIKE, LOVE, FIRE, BEARISH, BULLISH.
- 21. The system shall allow users to save, bookmark, and retrieve saved posts.
- 22. The system shall allow users to block/unblock other users and search users for @mentions.

Friends Module

- 23. The system shall allow users to send, accept, decline, and remove friend connections.
- 24. The system shall display pending friend requests and accepted friends list.

Tournament (Oracle) Module

- 25. The system shall allow users to start and join tournaments with AI-generated 30-day market simulations.
- 26. The system shall allow tournament participants to buy and sell stocks within the tournament context.
- 27. The system shall maintain a live tournament leaderboard updated in real-time.
- 28. The system shall generate AI post-tournament performance analysis for each participant.

AI Chatbot Module

- 29. The system shall provide a multi-turn AI chatbot coach with context from the user's portfolio, watchlist, and live prices.
- 30. The system shall persist chat session history for 24 hours per session.
- 31. The system shall generate periodic (weekly/monthly) AI performance reviews.

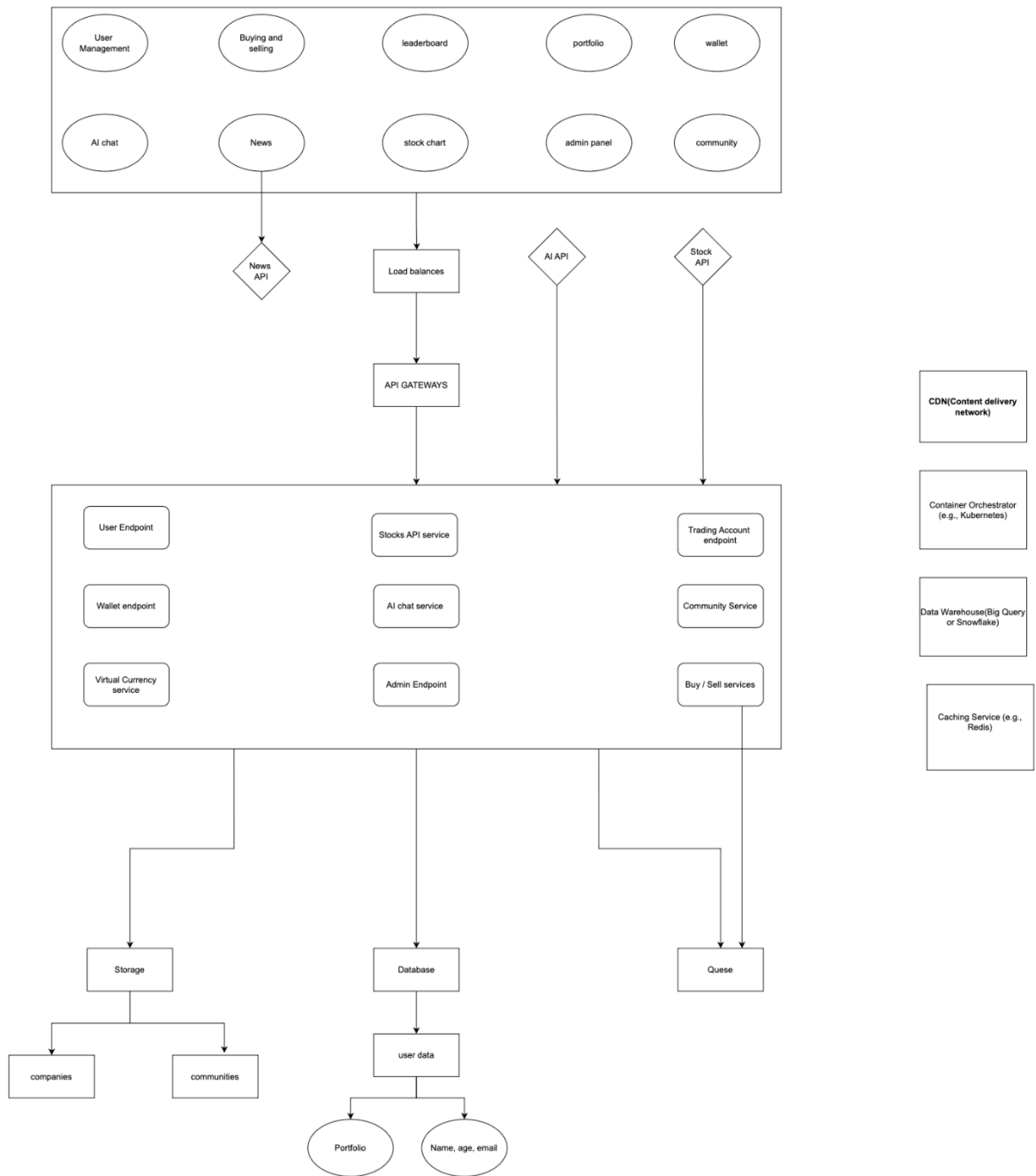
2c. Non-Functional Requirements

- 32. Memory: The system shall not exceed 4 GB total memory usage across all running services at any time.
- 33. Availability: The system shall maintain 99% uptime, with no more than 2 failures per 24-hour period. Recovery shall complete within 15 minutes of any failure.
- 34. Usability: The UI shall follow Jakob Nielsen's 10 usability heuristics and Ben Shneiderman's 8 golden rules.
- 35. Maintainability: The codebase shall follow clean code principles with modular, loosely-coupled components. Minor bug fixes shall be deployable within 48 hours; major fixes within 72 hours.
- 36. Real-time Performance: Stock price updates shall be delivered to connected clients within 2 seconds of a new tick under normal load conditions.
- 37. Scalability: The architecture shall support horizontal scaling of backend services to accommodate user base growth without redesign.
- 38. Security: The system shall comply with OWASP Top 10 security guidelines. All sensitive data shall be encrypted at rest and in transit via HTTPS/TLS.
- 39. Leaderboard Freshness: The global leaderboard shall be updated at the end of each market day.

3. System Architecture

Brief introduction of this chapter in a paragraph highlighting the content

a. Architecture Diagram



b. Architecture Description

- c. The system is composed of the following subsystems:

Frontend (Next.js 16 + React 19 + Tailwind 4): Serves all user-facing pages including dashboard, stock charts, portfolio, community, chatbot, and tournament views. Communicates with the backend over HTTP (REST) and WebSocket (Socket.IO). Deployed on Vercel.

Backend (NestJS 11): The core application server organized into feature modules: Auth, Users, Stocks, Watchlist, Trades, News, Friends, Community, Oracle (Tournament), Chatbot, and WebSocket gateways. Each module encapsulates its own controllers, services, and data access logic via Prisma ORM.

Database (PostgreSQL via Prisma 6): Primary relational data store for all user data, portfolios, transactions, community content, chat sessions, and tournaments. Schema managed via Prisma migrations.

Storage (Supabase): Used for user profile pictures and community post images. Provides signed URLs for secure access. Falls back to local storage when unavailable.

WebSocket Layer (Socket.IO): Two namespaces are maintained: /ws for real-time stock price ticks (subscribeSymbol/unsubscribeSymbol events), and /tournament for live tournament tick updates and leaderboard events.

External Services: PSX API (live and historical stock data), NewsAPI v2 (market news), Google Gemini API (chatbot, tournament generation, sentiment analysis, performance reviews).

d. Justification of the Architecture

- e. The modular monolith architecture deployed on Vercel (frontend) and a Node.js host (backend) is appropriate for TradeUp at its current stage because:

f.

Aspect	Discussion
Scalability	NestJS modules allow individual services (e.g., Chatbot, Oracle) to be extracted into microservices later without rewriting the application logic. Vercel auto-scales the frontend globally.
Maintainability	Clear module boundaries (each feature in its own NestJS module) align with clean code principles, enabling independent development and testing.
Security	JWT-based authentication on all protected routes, Prisma's parameterized queries mitigate SQL injection, and Supabase signed URLs protect file access.
Real-time Performance	Socket.IO WebSocket gateways provide sub-second tick delivery for stock prices and tournament updates, satisfying the real-time NFR.
Memory Efficiency	A single NestJS process runs all modules with shared resources; the Prisma connection pool limits DB connections, keeping memory well under 4 GB.

Con: Single Point of Failure	Unlike a microservices deployment, backend module failures affect the whole server. Mitigated by process managers (PM2) and platform restarts.
Con: Scaling Limits	Stateful WebSocket connections complicate horizontal backend scaling. Mitigated by Socket.IO Redis adapter in production.

g. Tools and Technologies

h. Category	Tool / Technology + Version
Backend Framework	NestJS 11
Frontend Framework	Next.js 16 (App Router), React 19
Programming Language	TypeScript (backend + frontend)
Database ORM	Prisma 6 (PostgreSQL)
Database	PostgreSQL 15
CSS Framework	Tailwind CSS 4
Authentication	JWT (jsonwebtoken), bcrypt
Real-time	Socket.IO (server + client)

AI Integration	Google Gemini API (gemini-1.5-flash)
File Storage	Supabase Storage (latest)
Candlestick Charts	lightweight-charts
Portfolio Charts	recharts
Form Validation	react-hook-form + zod
UI Animations	framer-motion
Toast Notifications	sonner
Dark/Light Mode	next-themes
UI Primitives	@radix-ui/* (dialog, dropdown, avatar, etc.)
Market Data API	PSX API (KSE-100 + listed stocks)
News API	NewsAPI.org v2
Version Control	GitHub
IDE	Windsurf (VS Code fork)
Frontend Deployment	Vercel
Cloud Infrastructure	AWS (ECS, S3, RDS, Route 53, CloudWatch)

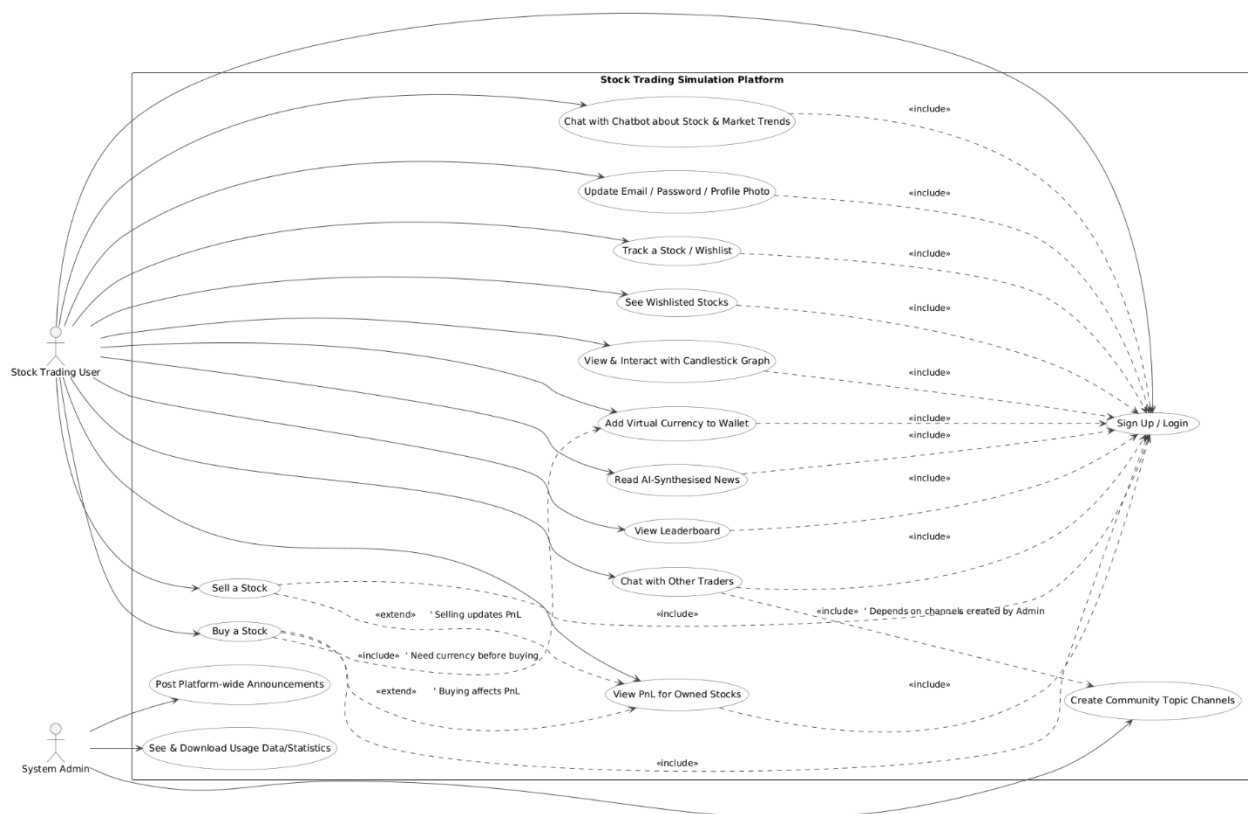
Containerization	Docker 27.0
Package Manager	npm

4. Requirements Specifications

This chapter provides the formal specification of system requirements through UML models. It covers the use case diagram and descriptions of 10 core use cases, the class diagram derived from the database schema, and sequence diagrams for the 10 core use cases.

a. Use Cases

Draw use case diagrams of your system using standard UML notation. Moreover, give description of 10 core use cases of your system



3.2.1 Update Email / Password / Profile Photo

Identifier	UC-001
Purpose	The user can update their account information (email, password, or profile photo).
Pre-conditions	The user is logged into their account and has completed authentication.
Post-conditions	The updated account information is saved successfully in the system.
Step #	Typical Course of Action
1	The user navigates to the profile settings page
2	The system displays editable fields for email, password, and profile photo.
3	The user selects which information to update (email/password/photo)
4	The user inputs the new information or uploads a new photo
5	The system validates the input (e.g., strong password, valid email format, correct image format).
6	The system prompts the user to confirm changes.
7	The user confirms the update.
8	The system updates the user's account information.
Step #	Alternate Courses of Action
	At step 5: If validation fails, the user is prompted to re-enter valid input before proceeding. At step 7: The user cancels instead of confirming → return to profile settings.
Step #	Exception Paths
	If the system cannot connect to the server at step 8, an error is shown and the update is not saved.

3.2.2 Track a Stock (Wishlist)

Identifier	UC-002
Purpose	The user can add a stock to their watchlist for monitoring.
Pre-conditions	The user is logged in and has access to the stock search feature.
Post-conditions	The selected stock is added to the user's watchlist.
Step #	Typical Course of Action
1	The user searches for a stock using the search bar.
2	The system retrieves stock information in real-time.
3	The user clicks "Add to Watchlist."
4	The system confirms the action and adds the stock to the watchlist.
Step #	Alternate Courses of Action
	At step 2: If no matching stock is found, the system displays "Stock not found."
Step #	Exception Paths
	If the server is unavailable during step 2, an error message is displayed and the process ends.

3.2.3 View PnL Information for Owned Stocks

Identifier	UC-003
Purpose	The user can view profit and loss (PnL) information for their owned stocks.
Pre-conditions	The user is logged in and has at least one stock in their portfolio.
Post-conditions	The system displays PnL calculations (realized/unrealized) for owned stocks.
Step #	Typical Course of Action
1	The user navigates to the “Portfolio” section.
2	The system retrieves current market data for all owned stocks.
3	The system calculates PnL values for each stock.
4	The system displays a summary table with stock names, current prices, buy prices, and PnL.
Step #	Alternate Courses of Action
	If the user has no owned stocks, the system displays a message: “You currently own no stocks.”
Step #	Exception Paths
	If market data cannot be retrieved at step 2, the system shows cached data with a warning: “Live data unavailable.”

3.2.4 Buying a specific stock

Identifier	UC-004
-------------------	--------

Purpose	The user can buy his desired stock
Pre-conditions	He has logged in his account and has done user authentication and has enough tokens to buy the stock.
Post-conditions	The stock is added in his portfolio and the information
Step #	Typical Course of Action
1	The user searches for the desired stock
2	The system retrieves and displays stock details (price, trend, charts).
3	The user clicks “Buy.”
4	The system prompts for quantity and confirms total cost
5	The user enters quantity.
6	The system verifies sufficient funds in wallet.
7	The system asks for confirmation of purchase
8	The user confirms.
9	The system deducts the amount from wallet and adds the stock to the portfolio
10	The use case ends
Step #	Alternate Courses of Action
	At step 5: The user cancels instead of entering a quantity → return to stock page. At step 7: The user cancels confirmation → return to stock page.
Step #	Exception Paths
	At step 6: If insufficient funds, the system shows an error and suggests adding money to wallet.

3.2.5 Stock selling

Identifier	UC-005
Purpose	The user can sell an owned stock to free up wallet balance
Pre-conditions	The user is logged in and owns the stock they wish to sell.
Post-conditions	The stock is removed (partially or fully) from the portfolio, and wallet balance is updated.
Step #	Typical Course of Action
1.	The user navigates to the “Portfolio” section.
2.	The system displays list of owned stocks.
3.	The user selects a stock and clicks “Sell.”
4.	The system prompts for quantity and shows current market price.
5.	The user enters quantity.
6.	The system confirms the sale details (quantity × price = total).
7.	The user confirms sale.
8.	The system updates wallet balance and removes or reduces the stock from the portfolio
Step #	Alternate Courses of Action
	At step 5: The user cancels → return to portfolio. At step 7: The user cancels confirmation → return to portfolio.

Step #	Exception Paths
	If user enters a quantity greater than owned, the system displays an error and blocks the action. If market data is temporarily unavailable at step 4, cached data is shown with a warning

3.2.6 See Wishlisted Stocks

Identifier	UC-006
Purpose	The user can view all stocks they have added to their watchlist.
Pre-conditions	The user is logged in and has previously added at least one stock to watchlist.
Post-conditions	The system displays real-time information for all wishlisted stocks.
Step #	Typical Course of Action
1.	The user navigates to “Watchlist.”
2.	The system retrieves list of wishlisted stocks.
3.	The system fetches real-time market data for each stock.
4.	The system displays stock list with current price, daily change, and trend indicators.
Step #	Alternate Courses of Action
	If the watchlist is empty, the system displays “No stocks in your watchlist” and suggests adding some.

Step #	Exception Paths
	If real-time data retrieval fails at step 3, cached data is displayed with a warning.

3.2.7 View and Interact with Candlestick Graph

Identifier	UC-007
Purpose	The user can view and interact with candlestick charts of stocks to analyze market trends.
Pre-conditions	The user is logged in and selects a stock that supports candlestick chart data.
Post-conditions	The candlestick chart is displayed with interactive options (zoom, timeframe change, hover tooltips).
Step #	Typical Course of Action
1.	The user selects a stock from portfolio, watchlist, or search results.
2.	The system retrieves stock's historical price data.
3.	The candlestick chart is generated and displayed.
4.	The user hovers over a candlestick to view open, close, high, and low values.
5.	The user can zoom in/out or switch timeframes (1D, 1W, 1M, etc.).
6.	The chart refreshes dynamically to reflect the chosen settings.
Step #	Alternate Courses of Action
	At step 5: If the user selects an unsupported timeframe, system displays "Timeframe not available."

Step #	Exception Paths
	At step 2: If historical data cannot be retrieved, system shows error “Chart data unavailable” and exits.

3.2.8 Add Money to Wallet

Identifier	UC-008
Purpose	The user can top up their simulation wallet with virtual funds for trading
Pre-conditions	The user is logged in and has access to wallet features
Post-conditions	The wallet balance is updated with the new funds.
Step #	Typical Course of Action
1.	The user navigates to the “Wallet” section.
2.	The system displays current wallet balance.
3.	The user clicks “Add Funds.”
4.	The system prompts for the top-up amount.
5.	The user enters the amount.
6.	The system confirms the new balance.
7.	The user confirms.
8.	The wallet is updated with the additional funds.
Step #	Alternate Courses of Action

	At step 5: If user cancels, return to wallet dashboard. At step 7: If user cancels, no funds are added.
Step #	Exception Paths
	If user enters a negative or invalid amount at step 5, an error message is displayed and input must be corrected.

3.2.9 Chat with Chatbot about Stock and Market Trends

Identifier	UC-009
Purpose	The user can interact with an AI chatbot to ask about stocks and market trends.
Pre-conditions	The user is logged in and chatbot service is active.
Post-conditions	The chatbot provides responses based on market data, insights, or educational content.
Step #	Typical Course of Action
1.	The user opens the “Chatbot” feature.
2.	The system displays chat interface.
3.	The user types a question (e.g., “What’s the trend for Tesla stock?”).
4.	The chatbot retrieves relevant stock data and AI insights.
5.	The chatbot generates a natural language response.
6.	The system displays the response in chat.
7.	The user can continue chatting or exit.
Step #	Alternate Courses of Action

	At step 3: If user asks a general trading question, chatbot responds with educational or help content instead of stock-specific data.
Step #	Exception Paths
	At step 4: If chatbot service is unavailable, the system displays “Chatbot currently offline.”

3.2.10 Read AI-Synthesised News

Identifier	UC-0010
Purpose	The user can read AI-generated news summaries and sentiment analysis on market events.
Pre-conditions	The user is logged in and the AI news feed service is active.
Post-conditions	AI-synthesised news articles and insights are displayed to the user
Step #	Typical Course of Action
1.	The user navigates to the “News” section
2.	The system fetches real-time financial and market news.
3.	The AI processes and summarizes key articles.
4.	The AI performs sentiment analysis (positive, neutral, negative).
5.	The system displays news summaries with sentiment indicators.
6.	The user clicks on a summary to view details.
7.	The system shows extended insights and related educational notes.

Step #	Alternate Courses of Action
	If user filters for a specific sector/stock, the system shows only relevant AI-synthesised news.
Step #	Exception Paths
	If external news sources are unavailable at step 2, the system displays cached news or “No updates available.”

b. Class Diagram

Use standard UML notation to draw the class diagram. Give brief description/purpose of each class in the class diagram. Give readable names to classes, attributes and operations.

Class	Description
User	Central entity. Attributes: id, email, username, passwordHash, role, balance, name, gender, profileImageUrl, tournamentScore. Operations: authenticate(), updateProfile(), addFunds().
Stock	Represents a PSX-listed stock. Attributes: id, symbol, name, marketType.
WatchlistItem	Association between User and Stock for watchlisting. Attributes: id, userId, symbol.
Portfolio	User's holding of a specific stock. Attributes: id, userId, symbol, quantity, avgPrice.

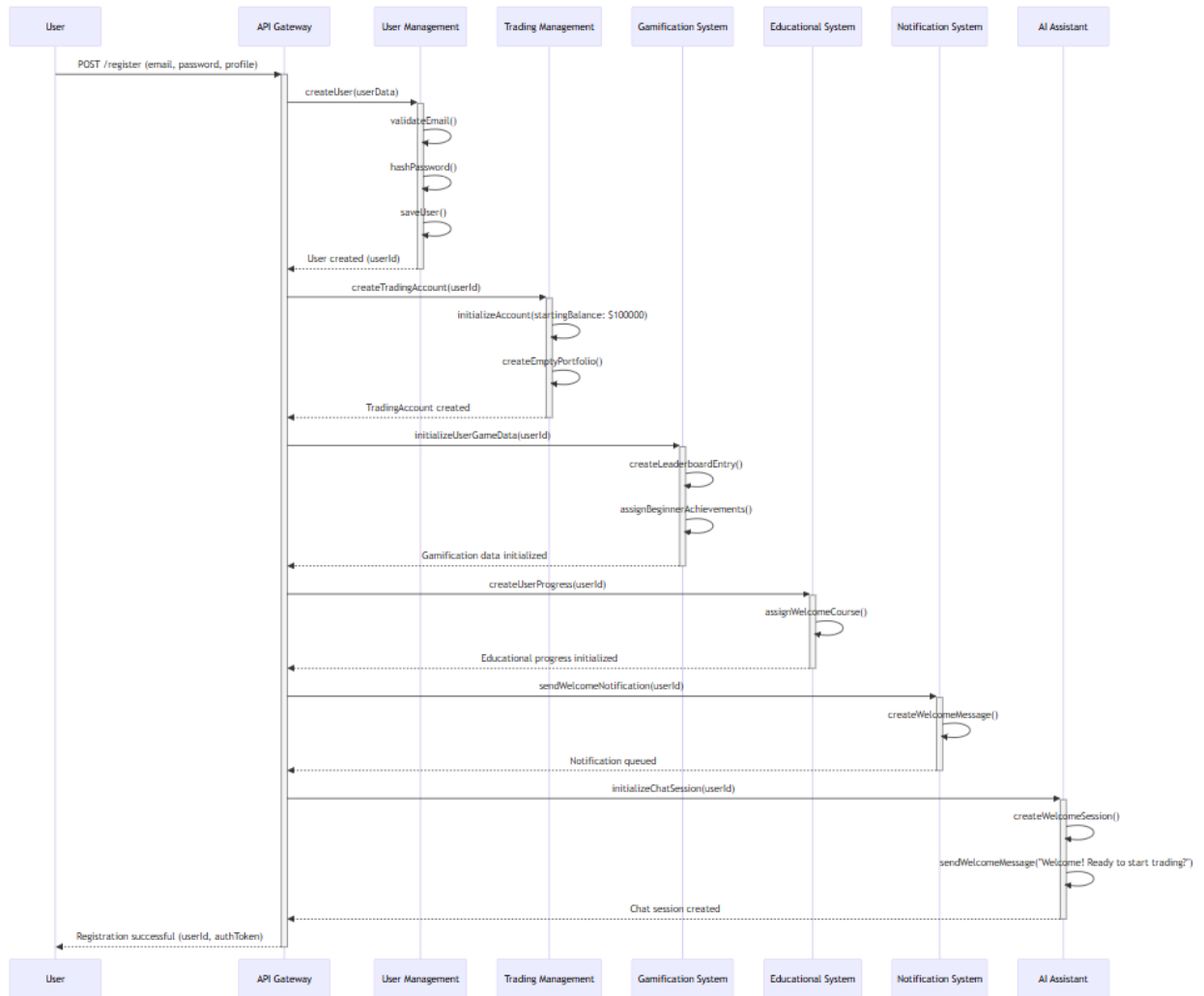
Transaction	Record of a buy or sell trade. Attributes: id, userId, type, symbol, quantity, price, total, sellReason, sellNote, createdAt.
Tournament	AI-generated 30-day simulation. Attributes: id, trajectoryJson, newsJson, status, startingCash, createdAt.
TournamentParticipant	User's enrollment in a tournament. Attributes: id, userId, tournamentId, balance, score.
TournamentPortfolio	Holdings within a tournament context. Attributes: id, participantId, symbol, quantity, avgPrice.
TournamentTransaction	Trade history within a tournament. Attributes: id, participantId, type, symbol, quantity, price, total.
Post	Community post. Attributes: id, authorId, title, content, tag, imageUrl, createdAt.
Comment	Nested reply on a post. Attributes: id, postId, authorId, content, parentId, createdAt.
Reaction	User reaction on a post. Attributes: id, userId, postId, type (LIKE/LOVE/FIRE/BEARISH/BULLISH).
SavedPost	Bookmarked post. Attributes: id, userId, postId, savedAt.
UserBlock	Block relationship. Attributes: id, blockerId, blockedId.

Friendship	Friend relationship. Attributes: id, requesterId, receiverId, status (PENDING/ACCEPTED/DECLINED).
ChatSession	24-hour chat window. Attributes: id, userId, createdAt, lastActive.
ChatMessage	Message within a session. Attributes: id, sessionId, role (user/assistant), content, createdAt.

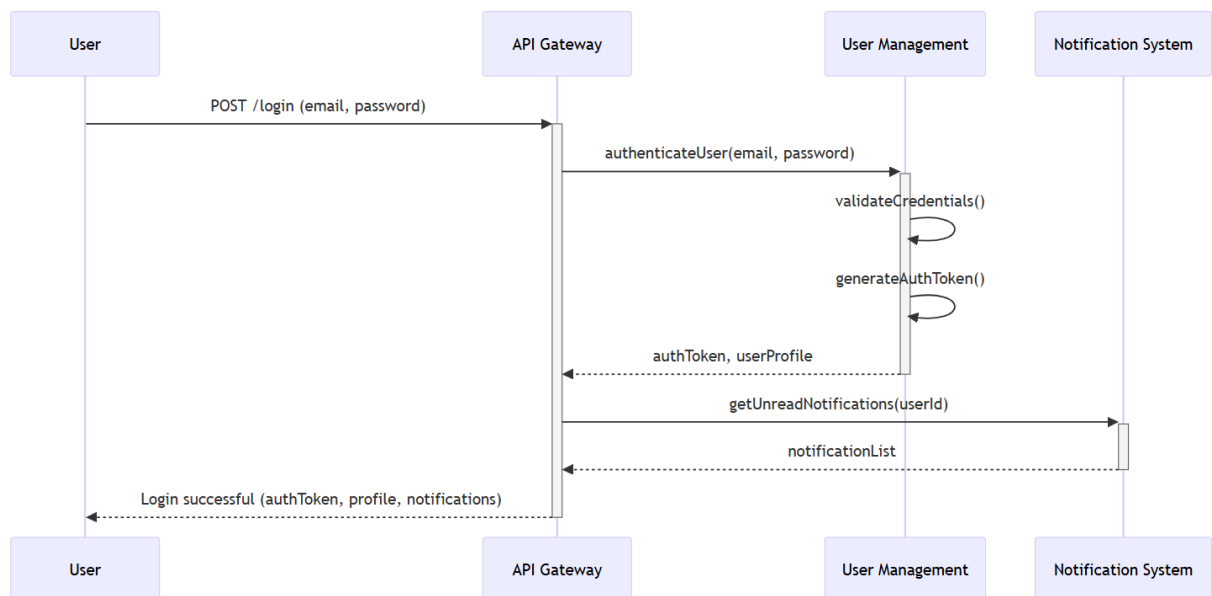
c. Sequence Diagrams

Draw sequence diagrams of 10 core use cases. Draw the diagrams using standard UML n

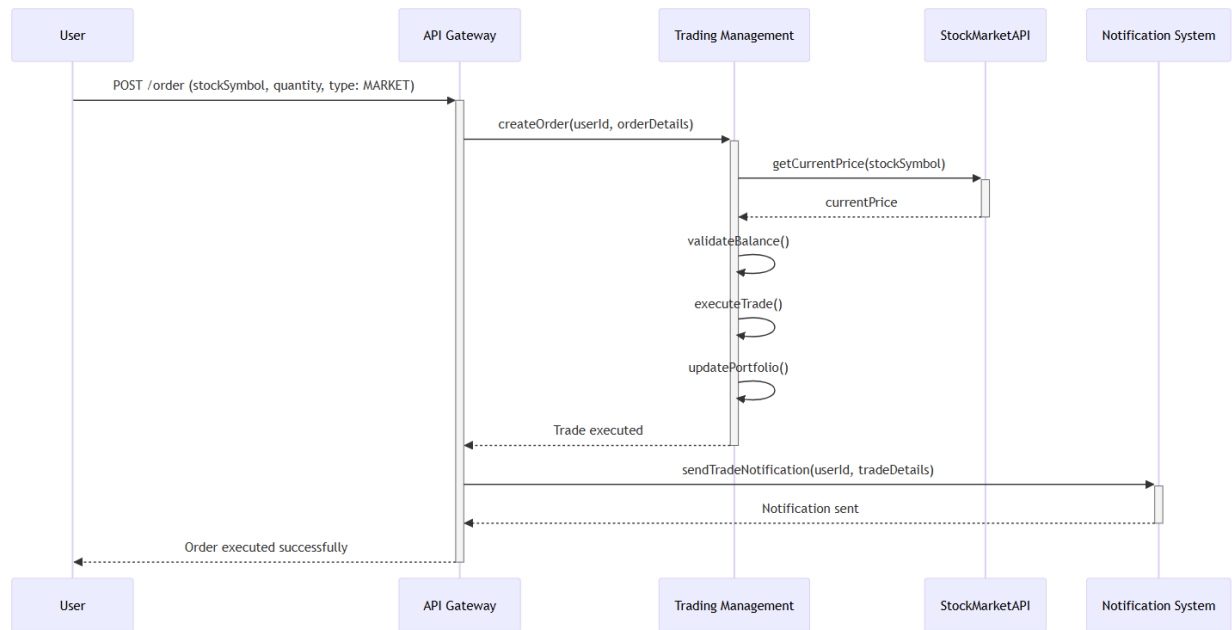
User Registration



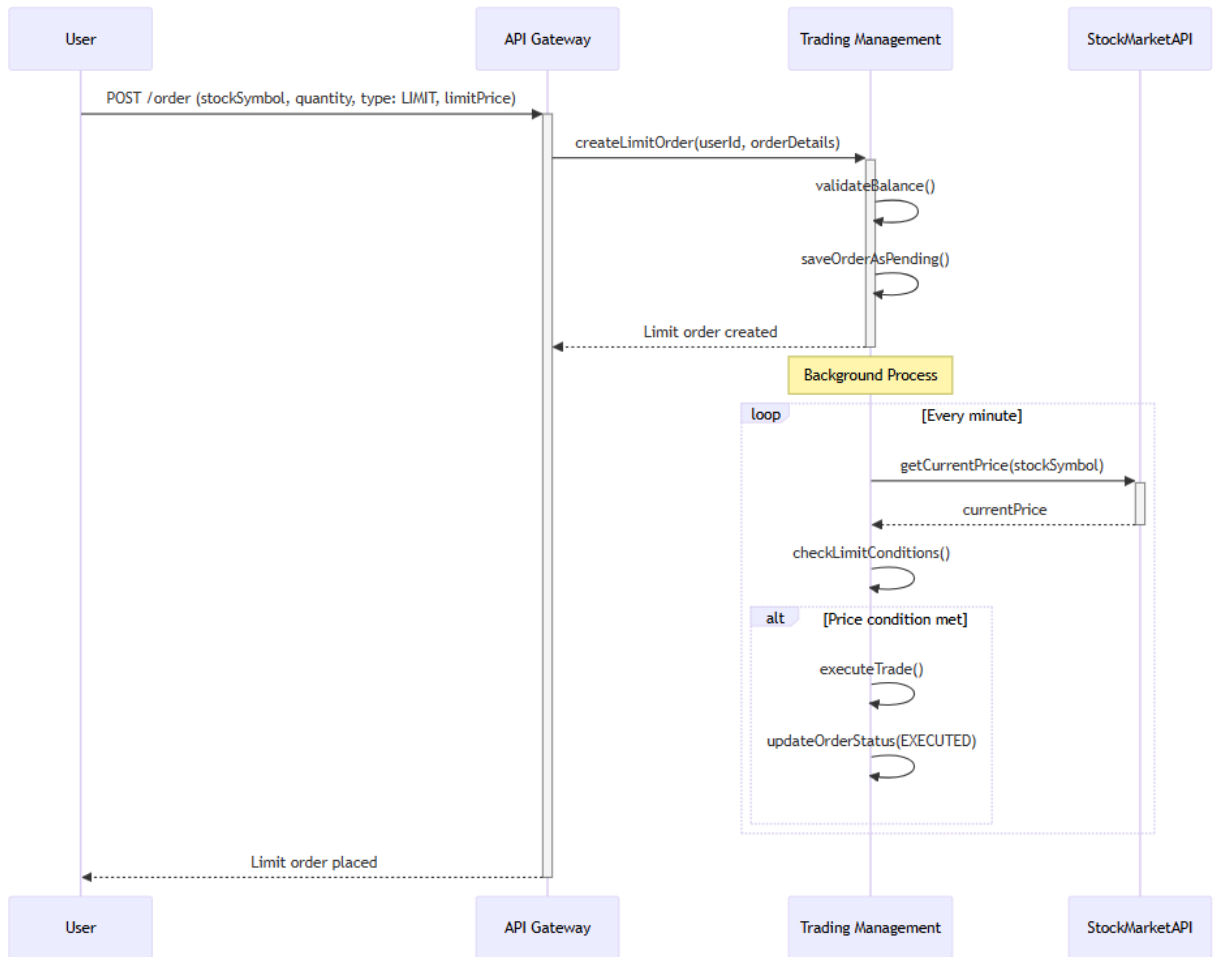
User login



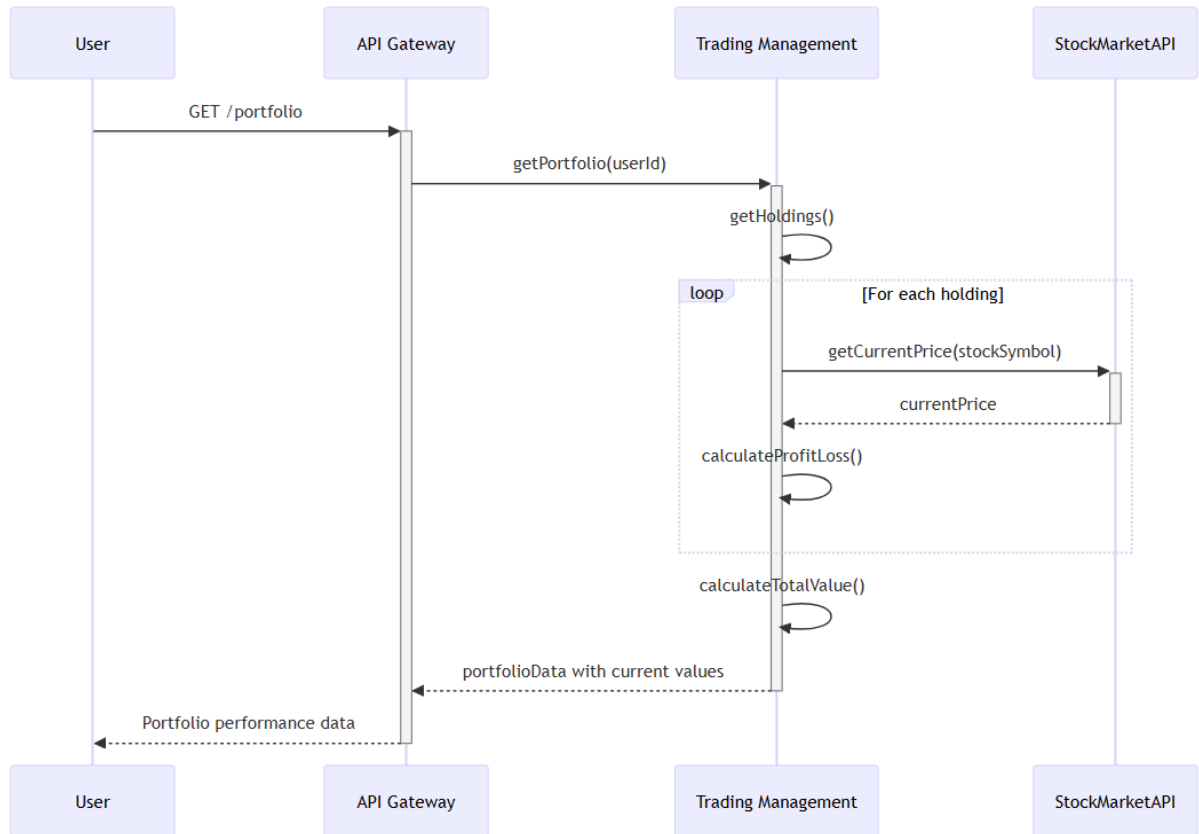
Place Market Order



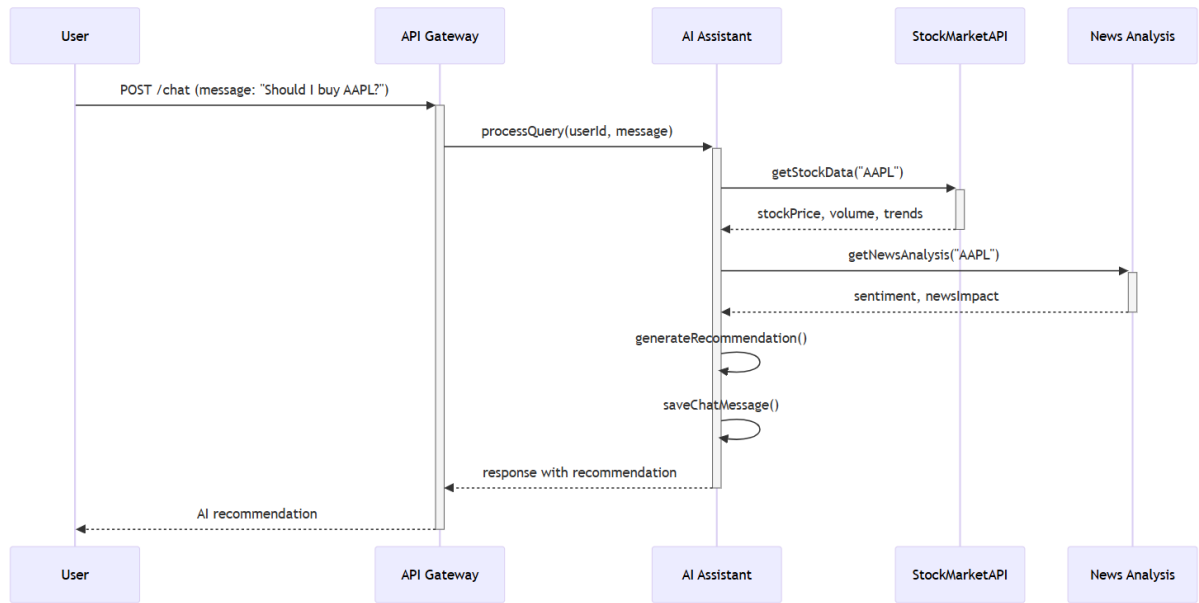
Place Limit Order



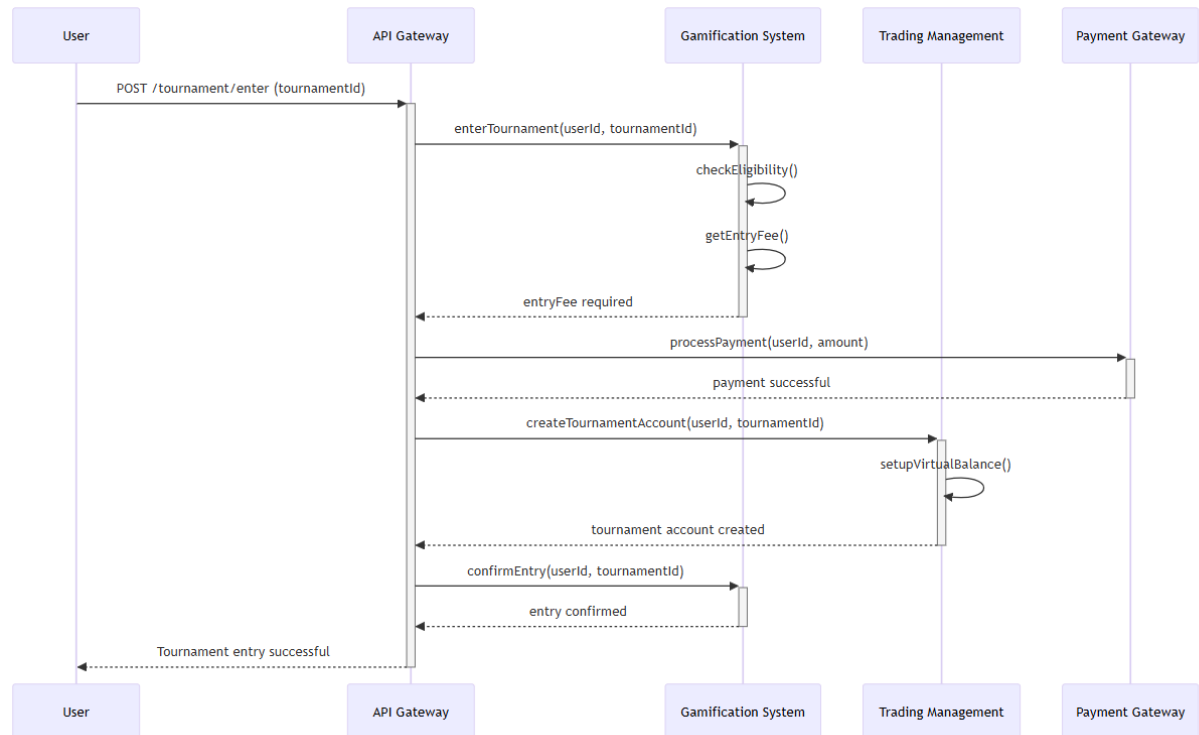
[View Portfolio](#)

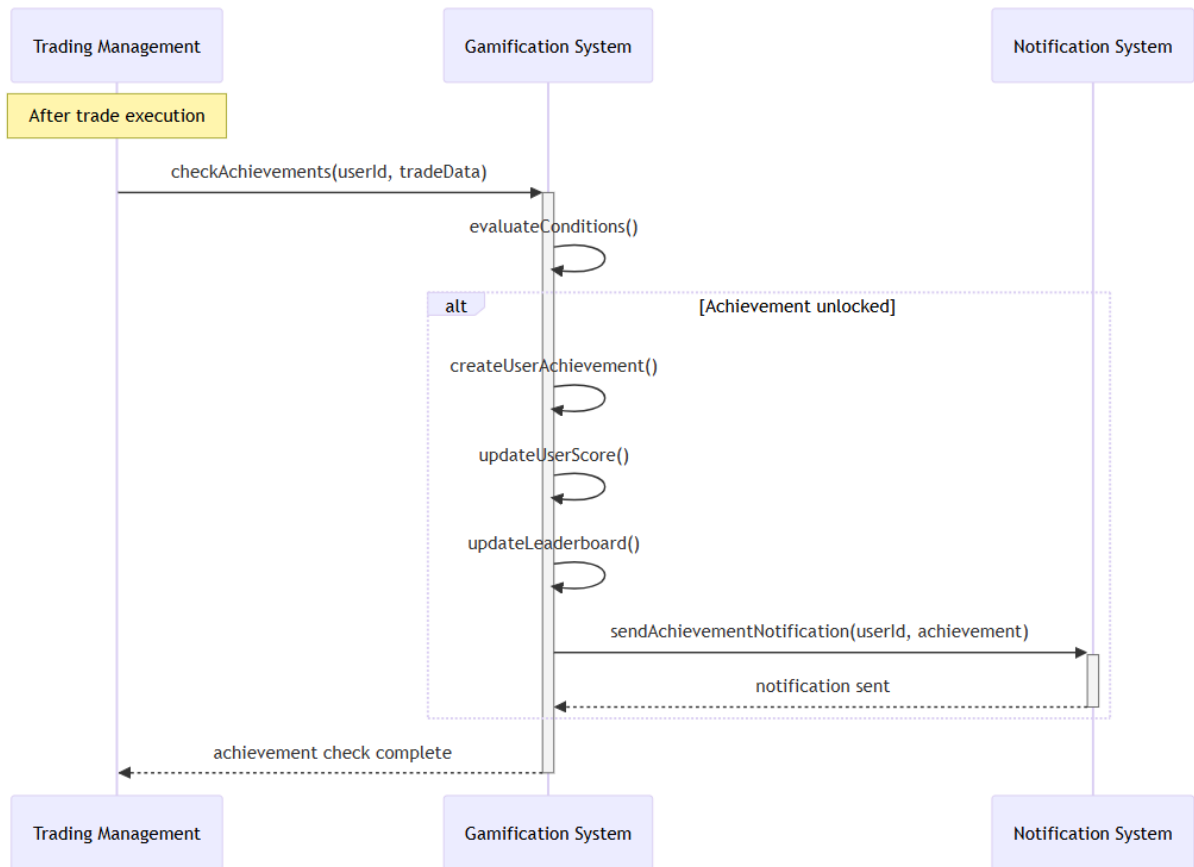


AI Assistant

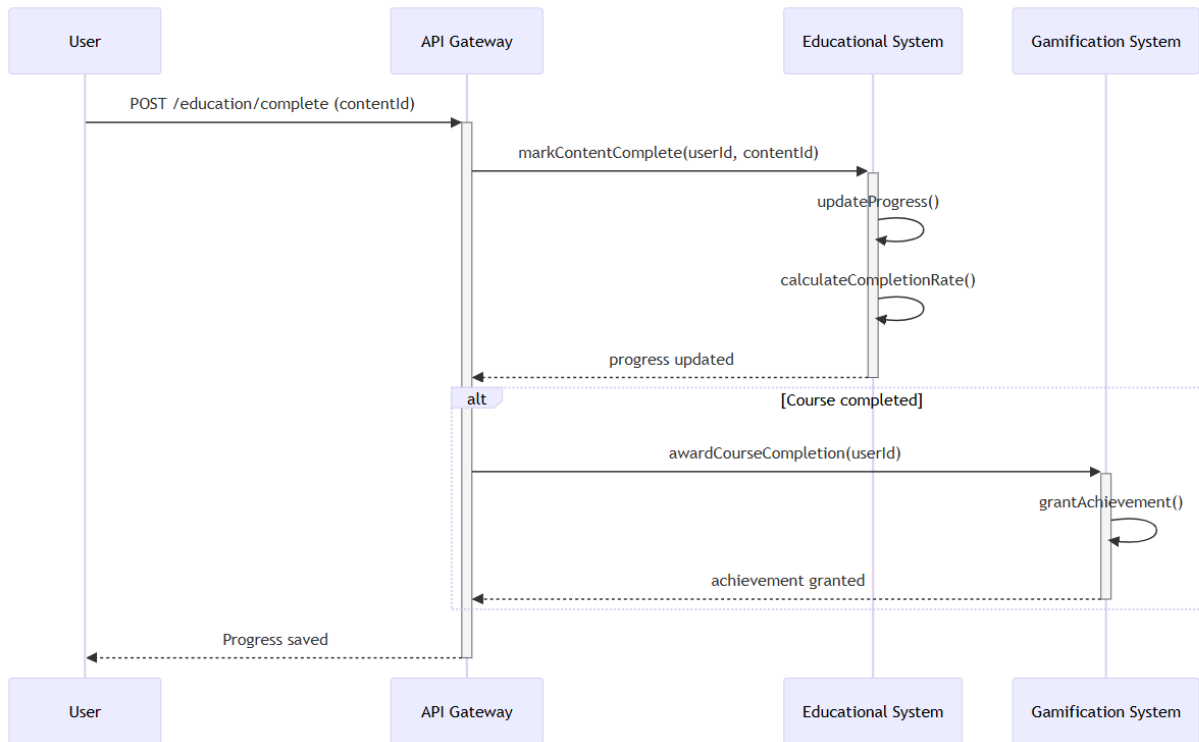


Game/Tournament

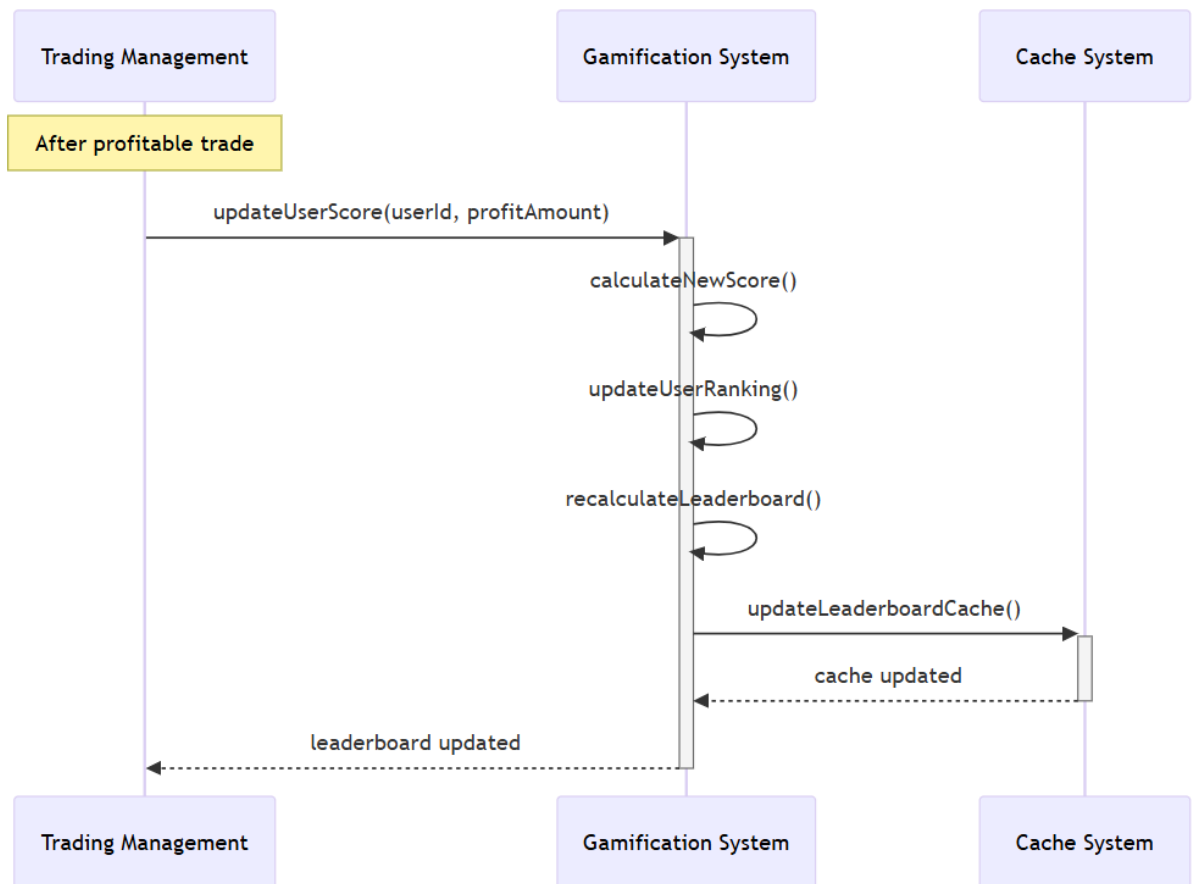




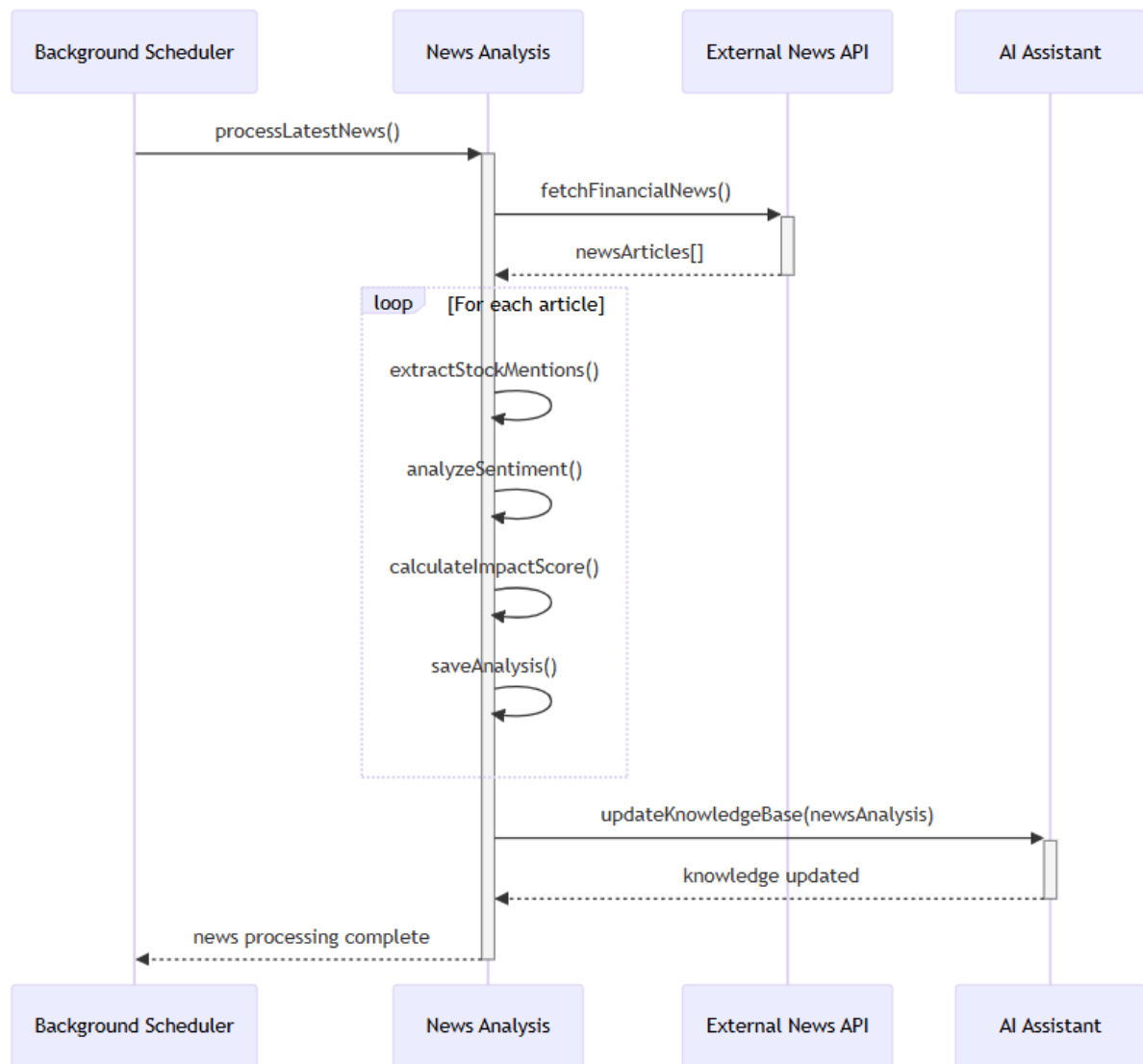
Educational content



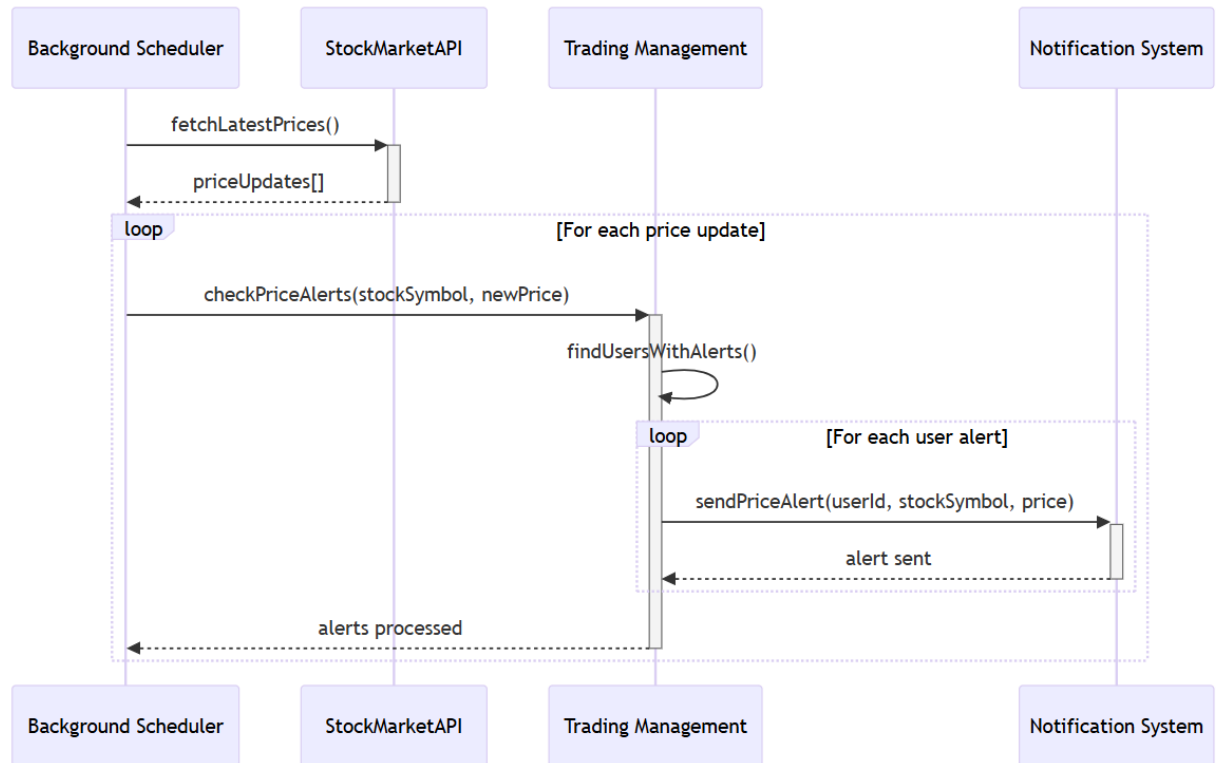
Leaderboard Update



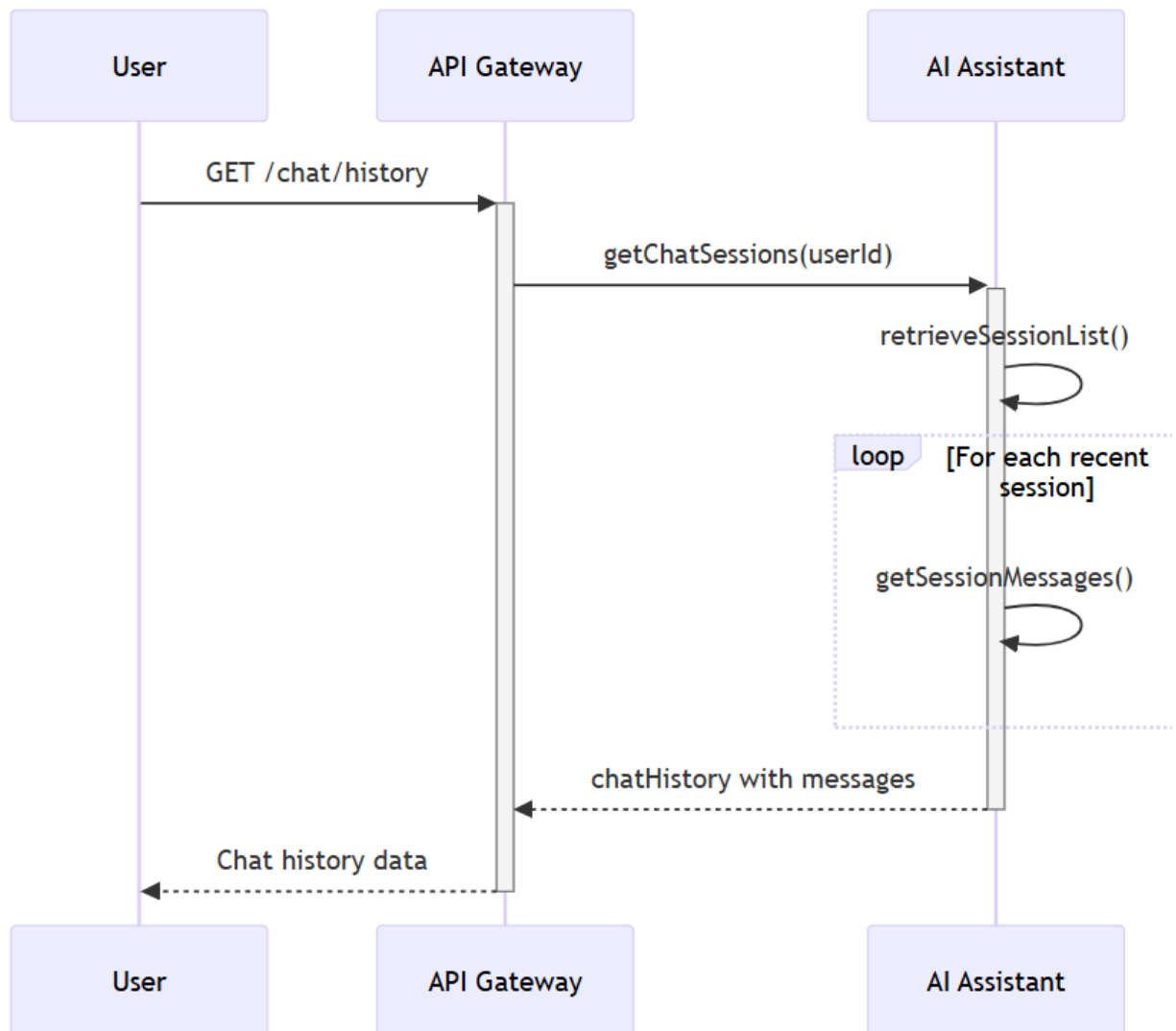
News Analysis



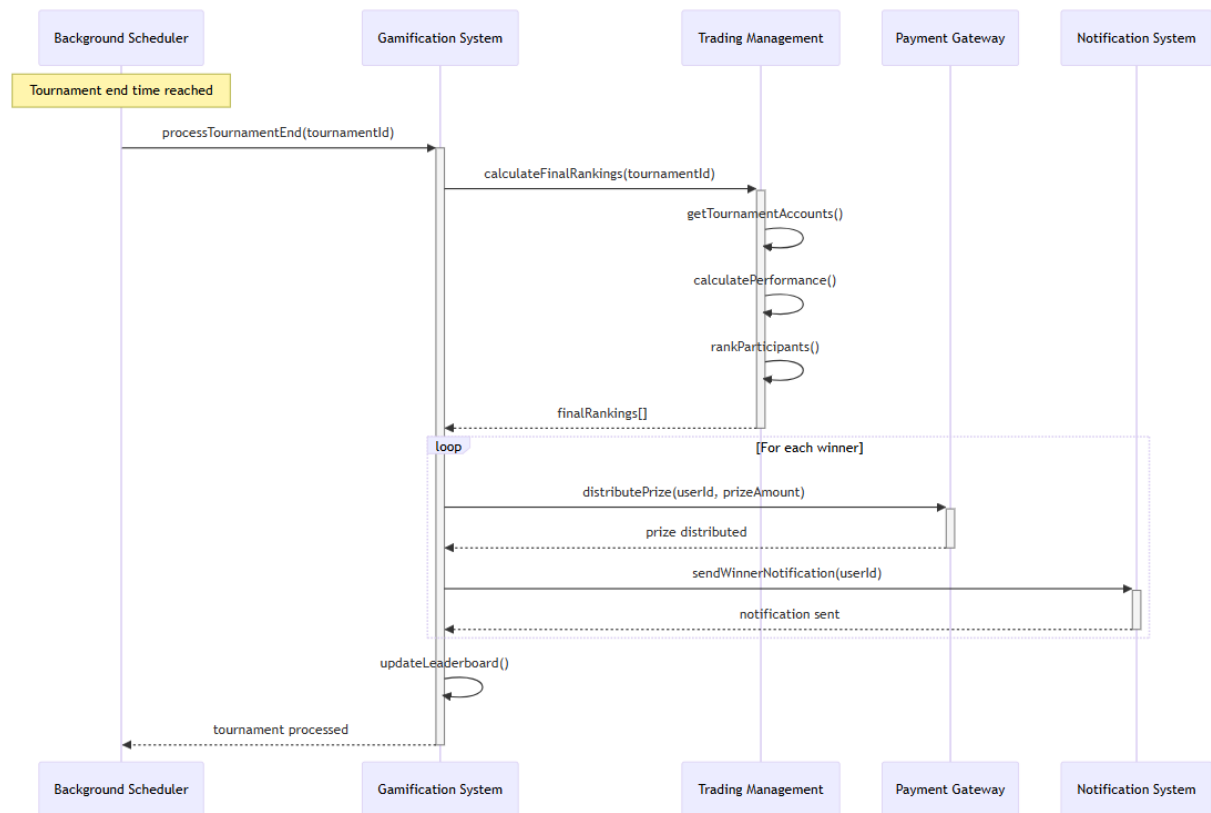
Stock Price Alert



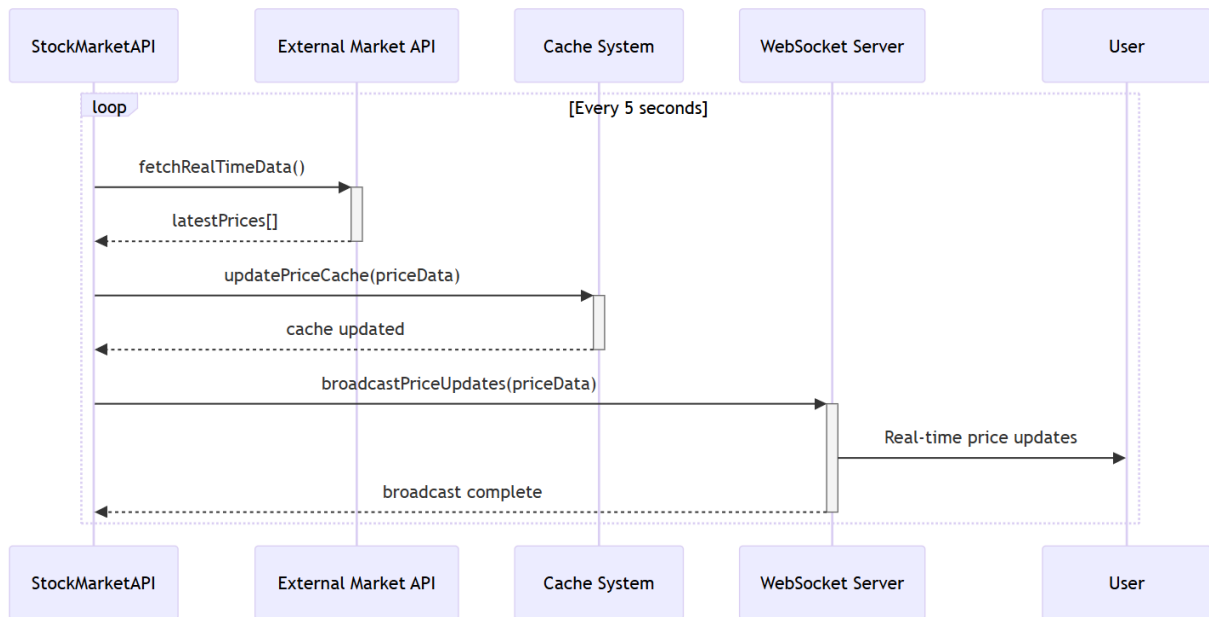
Chat



Tournament Results



Real time stock data sync



5. Software Development Methodology and Plan

Brief introduction of this chapter in a paragraph highlighting the content

a. Software Process Selection

Discuss a pros and cons of waterfall and agile (scrum) processes in your own words.

Give proper justification of the software development process that you have used for your project.

This chapter discusses the software development process selection for TradeUp. It evaluates Waterfall and Agile (Scrum) methodologies, justifies the chosen approach, and outlines how the sprints were structured.

5a. Software Process Selection

Waterfall

Waterfall is a linear, sequential development model where each phase (Requirements -> Design -> Implementation -> Testing -> Deployment) must complete before the next begins.

Pros: Clear milestones and deliverables at each phase. Easy to manage for well-understood, fixed-scope projects. Extensive upfront documentation reduces ambiguity during implementation.

Cons: Inflexible to changing requirements — going back to a previous phase is costly. Working software is only produced near the end, making early feedback impossible. In projects where requirements evolve (e.g., discovering that the PSX API returns data in an unexpected format), Waterfall provides no mechanism to adapt gracefully.

Agile (Scrum)

Scrum is an iterative, incremental framework organized around time-boxed sprints (typically 2-4 weeks). Each sprint delivers a potentially shippable product increment.

Pros: Highly adaptable to changing requirements. Continuous delivery of working software. Early and regular stakeholder feedback. Team velocity improves across sprints as the codebase matures. Retrospectives identify process improvements.

Cons: Scope creep is a risk if the product backlog is poorly managed. Requires active stakeholder engagement throughout. Documentation may be deprioritized in favor of working software. Difficult to estimate total project timeline upfront.

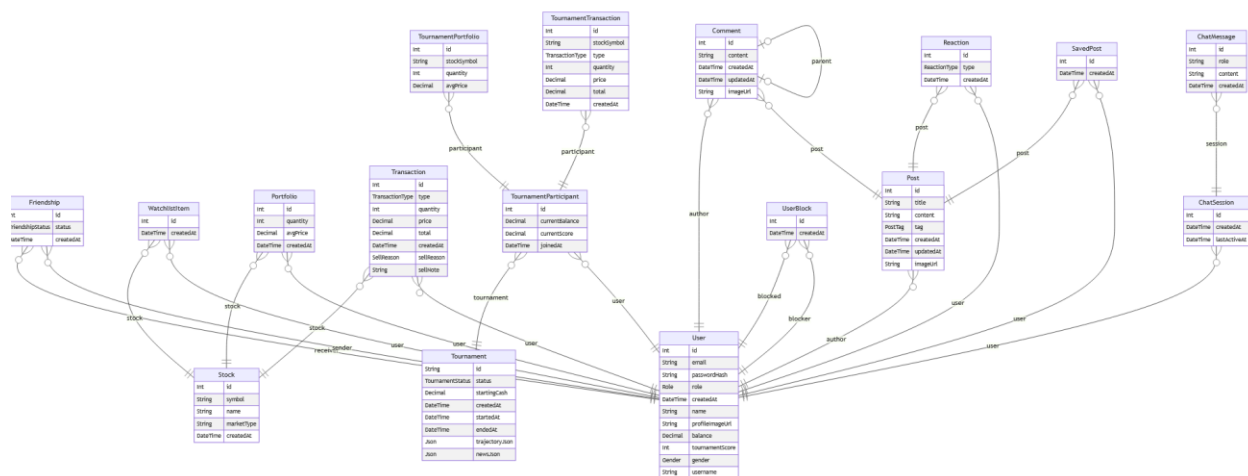
Justification

TradeUp used Agile Scrum with four sprints because the project's requirements evolved significantly during development. Initial plans included educational content and notification

systems that were later deprioritized in favor of the AI Tournament system — a feature whose technical complexity only became clear after the first sprint. Scrum allowed the team to re-prioritize the backlog between sprints based on technical feasibility, available APIs (PSX API behavior was only fully understood after Sprint 1 integration), and user feedback from informal demos. Each sprint included a security deliverable (STRIDE threat modeling for that sprint's use cases), ensuring security was baked in rather than bolted on.

6. Database Design and Web Services

Brief introduction of this chapter in a paragraph



The database is a PostgreSQL relational database managed via Prisma ORM. The schema is organized into four logical groups: Core Trading, Tournament, Community, and Chat.

[E/R Diagram — to be drawn showing entities and relationships as described below. Key relationships: User 1--* Portfolio, User 1--* Transaction, User 1--* WatchlistItem, User *--* Tournament (via TournamentParticipant), User 1--* Post, Post 1--* Comment, Post 1--* Reaction, User *--* User (via Friendship), User 1--* ChatSession, ChatSession 1--* ChatMessage]

a. Database Design

Create data model of your system. For example, E/R diagram. Give brief description of entities and data fields.

Entity	Key Fields and Purpose
User	id (UUID), email (unique), username (unique), passwordHash, role (USER/ADMIN), balance (Decimal), name, gender, profileImageUrl, tournamentScore. Central entity linked to all user-specific data.
Stock	id, symbol (unique PSX ticker), name, marketType. Lookup table for PSX-listed stocks.
WatchlistItem	id, userId (FK), symbol. Tracks which stocks a user is monitoring.
Portfolio	id, userId (FK), symbol, quantity, avgPrice (Decimal). Tracks current holdings and average cost basis.

Transaction	id, userId (FK), type (BUY/SELL), symbol, quantity, price, total, sellReason, sellNote, createdAt. Full trade history with journaling fields.
Tournament	id, trajectoryJson (AI-generated price paths), newsJson (AI-generated news timeline), status (ACTIVE/ENDED), startingCash, createdAt.
TournamentParticipant	id, userId (FK), tournamentId (FK), balance, score. Player state within a specific tournament.
TournamentPortfolio	id, participantId (FK), symbol, quantity, avgPrice. Holdings in tournament context (symbol-based, not FK to Stock).
TournamentTransaction	id, participantId (FK), type, symbol, quantity, price, total. Tournament trade log.
Post	id, authorId (FK), title, content, tag (enum), imageUrl, createdAt. Community post with optional media.
Comment	id, postId (FK), authorId (FK), content, parentId (self-referential for nesting), createdAt.
Reaction	id, userId (FK), postId (FK), type (LIKE/LOVE/FIRE/BEARISH/BULLISH). Toggle reaction model.
SavedPost	id, userId (FK), postId (FK), savedAt. Bookmarked posts.

UserBlock	id, blockerId (FK), blockedId (FK). Asymmetric block relationship.
Friendship	id, requesterId (FK), receiverId (FK), status (PENDING/ACCEPTED/DECLINED). Bidirectional friend graph.
ChatSession	id, userId (FK), createdAt, lastActive. 24-hour chat window tracking.
ChatMessage	id, sessionId (FK), role (user/assistant), content, createdAt. Persisted conversation history.

b. API Specification

This sub-section will contain the list of external APIs' that you have used in your project.

External API	Usage
PSX API (stock data provider)	Fetches live KSE-100 index data, PSX-listed stock prices, and historical OHLCV candle data. Used by the Stocks module and WebSocket tick engine.
NewsAPI.org v2	Retrieves latest market news and stock-specific news articles. API key stored in backend environment. Responses are passed to the frontend and optionally to the Gemini sentiment analysis endpoint.

Google Gemini API (gemini-1.5-flash)	Powers three backend services: (1) Chatbot: multi-turn conversational AI coach with portfolio context. (2) Oracle: generates 30-day tournament trajectoryJson and newsJson. (3) News Sentiment: classifies headlines with polarity, confidence, and keywords.
Supabase Storage API	Used for profile picture and community image uploads. Returns signed URLs for secure frontend access. Falls back to local storage if unavailable.

7. System User Interface

This sub-section should explain the core functionality of your application to the end user with supporting screenshots.

This section describes the core screens and user flows of TradeUp, explaining what each view does and how it connects to the system's functionality. [Screenshot placeholders are provided — add actual screenshots before submission.]

- **Dashboard / Home:** Shows the KSE-100 live index, featured PSX stocks with live price ticks, latest market news, and quick access to portfolio and watchlist. Real-time WebSocket updates are visible as prices fluctuate.
- **Buy/Sell Stock Page:** Displays a candlestick chart (multiple timeframes via lightweight-charts), live price, and a buy/sell form. Users enter quantity, see the estimated total, and submit. Sell form additionally prompts for a reason tag and optional note.
- **Portfolio Page:** Shows all current holdings with quantity, average price, current price, P&L (absolute and percentage), and overall portfolio value. Recharts components render a pie chart (allocation) and line chart (portfolio value over time).
- **Transaction History:** Paginated table of all past buy/sell trades, filterable by date. Each sell record displays the sell reason and note from the trade journal.
- **Watchlist Page:** Lists user-added stocks with live prices. Quick-add and remove controls.
- **News Feed:** Displays latest market headlines. Each headline has a 'Sentiment Analysis' button that calls the AI endpoint and displays polarity, confidence, and keywords inline.
- **Community Feed:** Posts organized by tag. Users can create posts (with optional image upload), comment in nested threads, react with 5 reaction types, save posts, and @mention other users with autocomplete. Blocked users' content is hidden.

- AI Chatbot: A chat interface with session persistence. The AI coach has full awareness of the user's portfolio, watchlist, and live prices. Users can ask trade-related questions and receive personalized advice.
- Tournament (Oracle) Page: Shows active tournament status, current simulated prices with AI-generated news events, buy/sell interface, live leaderboard, and personal tournament portfolio. Post-tournament screen shows the AI performance analysis.
- Profile / Settings: Allows users to update display name, email, password, and profile picture. Shows wallet balance with a 'Add Funds' option.

8. Project Security

This chapter details the security analysis performed on the TradeUp system using the STRIDE threat modeling framework, documents the mitigations implemented, maps threats to the OWASP Top 10, and describes the use of static analysis tooling.

8a. STRIDE Threat Modeling Summary

Threat modeling was performed across all four development sprints, covering each sprint's use cases before implementation began. The STRIDE framework was applied to each use case, analyzing threats across six dimensions: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege.

Sprint 1 covered: Update email/password, Add profile picture, P&L data visibility, Top-up wallet, AI stock simulation, View stock news. Sprint 2 covered: Sentiment analysis, Local/international news feeds, User search and add friends, View other users' portfolios, AI stock simulation model. Sprint 3 covered: Reactions/likes, Block/mute users, Community posts, Chart studying tools, Trading from chart view, AI game mode (tournament), Comments/replies, NLP order feedback, Realistic transaction delay, Visualize portfolio data, Trade journaling, Save community posts, Mention users, Post chart images.

8b. Key Security Controls Implemented

Control	Implementation
Authentication	JWT tokens issued on login, validated on every protected route via NestJS Guards. HttpOnly + Secure + SameSite cookies for session management.
Authorization (RBAC)	Role-based access control implemented via NestJS decorators. Admin-only endpoints protected by role check. Users can only access their own portfolio, transactions, and journal entries.
Input Validation	Zod schema validation on frontend forms. NestJS Pipes with class-validator for server-side DTO

Cryptographic Security	validation. Prisma's parameterized queries prevent SQL injection. Passwords hashed using bcrypt. JWT tokens signed with a secret stored in environment variables. All API communication over HTTPS/TLS. Supabase signed URLs for file access.
API Key Security	All third-party API keys (Gemini, NewsAPI, PSX API, Supabase) stored as environment variables, never hardcoded. In production, managed via AWS Secrets Manager.
Rate Limiting	Applied on auth endpoints (login, signup) and AI endpoints (chatbot, sentiment, oracle generation) to prevent abuse and DoS.
File Upload Security	Profile picture and post image uploads validated for MIME type server-side. Maximum file size enforced. Images stored in Supabase private buckets with signed URLs. EXIF metadata stripped before storage.
CORS Configuration	Restrictive CORS policy allowing only the registered frontend origin. Wildcard (*) not permitted in production.
Audit Logging	All sensitive actions (trades, account changes, wallet top-ups, tournament actions) logged with userId and timestamp.
Content Security Policy	CSP headers configured to block inline script execution, mitigating XSS risk in community post content.

8c. OWASP Top 10 Compliance

OWASP Category	Control in TradeUp
----------------	--------------------

A01 – Broken Access Control	Deny-by-default via NestJS Guards. Server-side ownership checks on every portfolio, trade, and journal endpoint. RBAC for admin routes.
A02 – Cryptographic Failures	bcrypt for passwords. HTTPS for all communication. Prisma encrypts database credentials. Supabase handles at-rest encryption.
A03 – Injection	Prisma ORM uses parameterized queries exclusively. Input sanitized via class-validator DTOs. API Gateway sanitizes inputs before routing.
A07 – Auth Failures	JWT-based stateless auth. Rate limiting on login. Password strength enforced at registration. Session tokens not exposed in URLs.
A09 – Security Logging Failures	Structured logging of all auth events, trades, and sensitive data access. Logs do not contain plaintext passwords or full JWT tokens.
ML02 – AI Data Poisoning	Gemini API called with sanitized, validated inputs. Prompt scrubbing removes PII before context injection. Rate limiting on AI endpoints.

8d. SonarQube / Static Analysis

Static code analysis was conducted using ESLint with NestJS and TypeScript rules on the backend, and Next.js ESLint config on the frontend. Both pipelines are run via `npm run lint` and integrated into the CI workflow on GitHub Actions. Any lint errors that indicate security-relevant issues (e.g., no-explicit-any, unsafe object access) are treated as blocking issues before merging to main.

9. Risk Management

Potential Risks and Mitigation Strategies

Sr.	Risk Description	Mitigation Strategy
1	Staff Turnover: A team member leaves before project completion.	Pre-risk: Cross-team code reviews and shared knowledge sessions. Require lead members to document designs. Post-risk: Next most experienced member steps up; leverage AI tools (under supervision) to fill gaps.
2	Requirements Change: Major design rework needed mid-project.	Identify which existing designs can be reused or adapted. Deprioritize non-core features. Maintain a clean backlog with labeled must-have vs. nice-to-have items.
3	Underestimation: System scope larger than expected.	Map all features to core value offerings (live trading, AI, community). Filter out non-essential features. Deliver a thinner but complete version of each core module.
4	Technology Change: Underlying technology superseded.	Assign one member to monitor technology changelogs. Use version-locked dependencies.

		Phase upgrades through a feature branch with full regression testing before merging.
5	AI Code Quality: AI-generated code is inefficient or insecure.	Use chain-of-thought prompting for AI code generation. Profile performance in dev. Review all AI-generated code for security issues before committing.
6	Data Unavailability: Required data (PSX API, NewsAPI) unavailable.	Implement mock data fallbacks for development. Cache API responses to reduce dependency on live feeds. Evaluate alternative data providers as backup.
7	Stakeholder Management: Clients misunderstand scope changes.	Translate technical impact into business terms (time, cost). Present in stakeholder meetings. Compromise only on non-core product value.
8	Off-the-shelf Library Issues: Libraries lack features or have defects.	Build missing functionality internally if critical. Leverage AI assistance for gap-filling. Evaluate alternative libraries early in the sprint.

9	Scalability Issues: System degrades under growing user load.	Load test early. Implement horizontal scaling via AWS ECS auto-scaling groups. Use Socket.IO Redis adapter for multi-instance WebSocket support. Monitor via AWS CloudWatch.
10	Data Privacy Breaches: User data exposed via vulnerability.	Strong encryption for storage and transit. JWT-based auth with short expiry. Incident response plan ready. Notify users and patch vulnerabilities immediately on discovery.

10. Testing and Evaluation

Discuss your testing strategy. List down some sample test cases that your created. Include some AI test cases as well.

Moreover, list down the automation tools you used.

This chapter describes the testing strategy adopted for TradeUp, sample test cases including AI-specific tests, and the automation tools used.

Testing Strategy

TradeUp employed a layered testing strategy:

- Unit Testing: Individual NestJS service methods tested in isolation using Jest. Mock implementations of Prisma client and external API calls used to isolate units.
- Integration Testing: API endpoints tested end-to-end using Supertest, spinning up the NestJS application in a test environment with a test PostgreSQL database.
- Manual Testing: Frontend flows tested manually across Chrome and Safari for key user journeys (registration, buy/sell, chatbot, tournament). Postman collections used for API endpoint validation.
- Security Testing: STRIDE threat modeling per sprint. ESLint security rules run in CI. Manual review of authentication flows and IDOR vulnerability scenarios.

Sample Test Cases

Test Case	Expected Outcome
TC-01: Register with valid inputs	201 response, JWT returned, user record created in DB with hashed password.
TC-02: Login with wrong password	401 Unauthorized response. No JWT issued.
TC-03: Buy stock with sufficient balance	200 response, portfolio updated, transaction logged, balance deducted.
TC-04: Buy stock with insufficient balance	400 Bad Request — insufficient balance error.
TC-05: Sell stock not in portfolio	400 Bad Request — holding not found error.
TC-06: Access protected endpoint without JWT	401 Unauthorized from NestJS Guard.
TC-07: Access another user's portfolio via ID manipulation	403 Forbidden — server-side ownership check prevents access.
TC-08: Upload file exceeding size limit	400 Bad Request — file size validation fails.
TC-09 (AI): Chatbot responds to portfolio query	AI response references the user's actual holdings and current prices, not generic advice.
TC-10 (AI): Tournament generates 30-day trajectory	Oracle endpoint returns valid trajectoryJson with 30 daily price entries per stock and corresponding newsJson events.

TC-11 (AI): Sentiment analysis of a bullish headline	API returns { sentiment: 'positive', confidence: > 0.7, keywords: [...] }.
TC-12: WebSocket tick subscription	After subscribing to a symbol, client receives tickUpdate events within the polling interval with valid price data.
TC-13: Rate limit on login endpoint	After N failed attempts within 1 minute, subsequent attempts return 429 Too Many Requests.
TC-14: SQL injection attempt in search	Input sanitized by Prisma ORM, no DB error, safe empty result returned.

Automation Tools

- Jest: Unit and integration test runner for NestJS backend.
- Supertest: HTTP assertion library for NestJS integration tests.
- Postman: Manual and automated API collection testing.
- ESLint (NestJS + TypeScript + Next.js configs): Static analysis in CI pipeline.
- GitHub Actions: CI workflow runs lint and test suite on every pull request to main.

This section provides step-by-step deployment instructions for TradeUp in a production environment, including CI/CD setup and Kubernetes scalability considerations.

Source Code

GitHub Repository: <https://github.com/Raiyaan17/P04-TradeUp> (private — request access from team).

Live Application: <https://p04-trade-up.vercel.app>

Test Credentials: Username: `testuser@tradeup.pk` | Password: `TestPass123!`

Step-by-Step Deployment

Step 1: Clone the Repository

- `git clone https://github.com/Raiyaan17/P04-TradeUp.git`
- `cd P04-TradeUp/Prototype`

Step 2: Configure Environment Variables

Backend — create `Prototype/backend/.env` with:

- `DATABASE_URL=postgresql://<user>:<password>@<host>:5432/<db>`
- `JWT_SECRET=<strong-random-secret>`
- `PORT=3001`
- `GEMINI_API_KEY=<your-gemini-key>`
- `SUPABASE_URL=<your-supabase-url>`

- SUPABASE_SERVICE_ROLE_KEY=<your-supabase-service-key>
- NEWS_API_KEY=<your-newsapi-key>
- STOCK_API_KEY=<your-psx-api-key>

Frontend — create Prototype/frontend/.env.local with:

- NEXT_PUBLIC_API_BASE_URL=https://<your-backend-domain>:3001

Step 3: Database Setup

- cd backend
- npm install
- npm run prisma:generate
- npm run prisma:migrate # runs all Prisma migrations

Step 4: Build and Run Backend

- npm run build
- npm run start:prod # production mode

Step 5: Build and Deploy Frontend

- cd ../frontend && npm install
- npm run build
- Deploy to Vercel: vercel --prod (or connect GitHub repo to Vercel dashboard)

Step 6: CI/CD Pipeline (GitHub Actions)

The `.github/workflows` directory contains workflows that: (1) Run ESLint on backend and frontend on every PR. (2) Run Jest test suite on backend on every PR. (3) Auto-deploy frontend to Vercel on merge to main via Vercel GitHub integration. (4) Build and push backend Docker image to AWS ECR on merge to main using AWS CodePipeline.

Application Scalability with Kubernetes

For large-scale production deployment, TradeUp's backend can be containerized (Dockerfile provided) and deployed to a Kubernetes cluster (e.g., AWS EKS). The NestJS backend is stateless for REST endpoints and can be horizontally scaled by increasing the replica count in the Kubernetes Deployment manifest. WebSocket scalability requires deploying a Redis instance and configuring the Socket.IO Redis adapter so that all backend pods share a common pub/sub channel for tick broadcasting. A Kubernetes HorizontalPodAutoscaler (HPA) can be configured to scale pods based on CPU utilization (e.g., target 60% CPU), automatically provisioning additional pods during market-open traffic spikes. An AWS Application Load Balancer Ingress Controller handles sticky sessions for WebSocket connections. The PostgreSQL database is deployed as AWS RDS with read replicas for query-heavy operations (portfolio reads, leaderboard queries). Kubernetes liveness and readiness probes ensure unhealthy pods are replaced within the 15-minute SLA defined in the non-functional requirements.

11. Deployment Guidelines

List down the steps for deployment of your system. Start from where the code (link of the Github repository) should be picked and then mention all the steps for deployment in a production environment. Also mention the online link where your application is hosted along with access information (user/password etc.). You may include CI/CD pipeline details for automated deployment. Also include a paragraph on application scalability using Kubernetes.

12. Conclusion

a. Summary

This sub-section will summarize how you have approached this project and what you have learned.

b. Challenges

Elaborate the issues and challenges, both technical and non-technical, faced during this project and how you have addressed them.

c. Future

Describe how the application can be extended in the future by you or someone else. Include any recommendations that you received from visitors on project showcase day.

13. Review checklist

Before submission of this report, the team must perform an internal review. Each team member will review one or more sections of the deliverable.

Chapter/Section Name	Reviewer Name(s)

14. References